CSE485/CSE488 Deep Learning

Assignment 1: MicroGrad (Scalar Autograd Engine)

Abdelmoneim Mahmoud (23P0015)

Branch: `abdelmoneim-mahmoud_23P0015` PR: *[add PR link]*

1 Overview

`src/engine.py` implements a `Value` class. It wraps one scalar, records the operation that produced it, and stores a gradient. `src/nn.py` builds `Neuron`, `Layer` and `MLP` on top of it. All 12 tests in the template pass (`python -m pytest`), and `demo.ipynb` trains an MLP on the two-moons dataset.

2 Mathematical Derivations

Each operation creates an output node $o = f(a, b)$. During backpropagation the node already holds $\frac{\partial L}{\partial o}$ (`out.grad`). The chain rule then gives the contribution to each input:

$$\frac{\partial L}{\partial a} \mathrel{+}= \frac{\partial o}{\partial a} \cdot \frac{\partial L}{\partial o}.$$

Each `_backward` closure only needs the *local* derivative $\partial o/\partial a$:

Operation	Forward	Local derivative(s)
Addition	$o = a + b$	$\partial o/\partial a = 1,\ \partial o/\partial b = 1$
Multiplication	$o = a\,b$	$\partial o/\partial a = b,\ \partial o/\partial b = a$
Power (n constant)	$o = a^n$	$\partial o/\partial a = n\,a^{n-1}$
Exponential	$o = e^a$	$\partial o/\partial a = e^a = o$
Tanh	$o = \tanh a$	$\partial o/\partial a = 1 - \tanh^2 a = 1 - o^2$

Tanh. $\tanh a = \frac{e^a - e^{-a}}{e^a + e^{-a}}$. The quotient rule gives $\frac{d}{da}\tanh a = \frac{(e^a+e^{-a})^2 - (e^a - e^{-a})^2}{(e^a+e^{-a})^2} = 1 - \tanh^2 a$. This is why the closure can reuse the forward output t and skip any extra exponentials.

Derived operations. These are built from the primitives above, so they need no backward code of their own:

$$-a = a \cdot (-1), \qquad a - b = a + (-b), \qquad a/b = a \cdot b^{-1}.$$

For division, the power rule gives $\frac{\partial}{\partial b}\, a\, b^{-1} = -a\, b^{-2}$ automatically. If $b = 0$, computing b^{-1} raises `ZeroDivisionError` in the forward pass, which is what the tests expect.

Why `+=`. If a node feeds into several parents, the multivariable chain rule says $\frac{\partial L}{\partial a} = \sum_k \frac{\partial L}{\partial o_k}\frac{\partial o_k}{\partial a}$. Each parent adds its own term, so the gradient must be accumulated (`+=`) and not overwritten. Example: for $d = (a + a)\,a = 2a^2$ we need $\partial d/\partial a = 4a = 12$ at $a = 3$.

3 Topological Sort and `backward()`

```
def _get_topo(self):
    topo, visited = [], set()
    def build_topo(node):
        if node not in visited:
            visited.add(node)
            for child in node._prev:
                build_topo(child)
            topo.append(node) # post-order
    build_topo(self)
    return topo

def backward(self):
    topo = self._get_topo()
    self.grad = 1.0 # dL/dL = 1
    for node in reversed(topo):
        node._backward()
```

Post-order. A node is appended only *after* all of its inputs. So in `topo` every node comes after the nodes it depends on. Reversing the list puts every node *before* its inputs. When a node's `_backward` runs, every parent that uses it has already finished, and its `grad` is complete. If we appended before visiting the children, a node could push its gradient down before receiving all of its own.

Visited set (diamond graph). Take $c = a + a$. The node a appears twice in the graph but must appear *once* in `topo`, otherwise its `_backward` would run twice and double-count. Trace: `build_topo(c)` visits a and appends it, so `topo`$= [a, c]$. Then c.`grad`$= 1$, and c.`_backward` adds 1 to a.`grad` once for each operand (`self` and `other` are both a), giving a.`grad`$= 2 = \frac{d(2a)}{da}$. Without the set, large graphs with shared nodes would also be revisited an exponential number of times. The set hashes nodes by identity. That is why `Value` does not override `__eq__`: two nodes with equal data must not collapse into one.

4 Design Justifications

- **Strict input types.** `__add__` and `__mul__` wrap only `int`/`float`/`Value` and raise `TypeError` otherwise (for example `Value(2) + "string"`). Strings and lists would otherwise produce silent nonsense.
- **Power with a constant exponent only.** `__pow__` asserts the exponent is `int`/`float`, as the template and tests require. A `Value` exponent would need $\partial(a^b)/\partial b = a^b \ln a$, which is undefined for $a \leq 0$.
- **Reverse operators.** `__radd__`, `__rmul__`, `__rsub__` and `__rtruediv__` make `2 + a`, `1 - a` and `2 / a` work. `__radd__` also lets Python's `sum()` start from `0`.
- **Closures for `_backward`.** Each closure captures its own `self`, `other` and `out`, so the graph needs no separate operation table.
- **Neural-network API.** `Neuron` computes $\tanh(\mathbf{w}\cdot\mathbf{x}+b)$ with $w_i, b \sim U(-1, 1)$. `Layer` returns a single `Value` when it has one neuron (this makes scalar losses easy). `parameters()` flattens recursively, and `zero_grad()` calls `reset_grad()` on every parameter.
- **Linear output layer.** `Neuron` has an optional `nonlin` flag (default `True`, so the tests behave exactly as specified). `MLP` turns it off for the last layer only. A tanh output is bounded in $[-1, 1]$, so the hinge loss $\max(0, 1 - y\,s)$ could never be fully satisfied and its gradient would

shrink near saturation. A linear output gives an unbounded score s and avoids this. The same design is used in Karpathy's micrograd and in standard classifiers (logits).

5 Conceptual Questions

Q1. Why reverse-mode differentiation for neural networks? Training has one scalar output (the loss) and many inputs (the parameters). Reverse mode computes *all* $\partial L/\partial\theta_i$ in one backward pass, costing about as much as the forward pass. Forward mode would need one pass per parameter (337 passes for our MLP, millions for real networks).

Q2. Why must `+=` be paired with `zero_grad()`? Accumulation is correct *within* one backward pass (shared nodes, Section 2). Across training steps, though, `grad` would keep the previous step's values, and the update would use a stale running sum. So `zero_grad()` is called before every `backward()`. Fresh intermediate nodes are rebuilt every forward pass, so only the parameters need resetting.

Q3. What happens to gradients when tanh saturates? $\tanh'(a) = 1 - \tanh^2 a \to 0$ as $|a|$ grows. In a deep stack these factors multiply, so gradients for early layers can vanish. Mitigations: small initial weights, normalised inputs, ReLU-type activations, normalisation layers.

Q4. What is the cost of a scalar engine compared with PyTorch? Every scalar operation is a Python object plus a closure. Our 100-sample, 337-parameter MLP builds a graph of about 69,000 nodes per step, and the notebook takes about 1–2 minutes for 100 steps. PyTorch represents a whole matrix multiply as *one* node with a tensor-level backward ($\partial L/\partial W = \delta\, x^\top$) running in optimised C++/CUDA. The maths is identical. Only the granularity of the nodes changes.

Q5. Why do we need the recursion limit in the demo? `_get_topo` is recursive, and its depth equals the longest path in the graph. Summing 100 losses and 337 squared parameters with `sum()` creates a chain several hundred nodes long, close to Python's default limit of 1000. The notebook raises the limit with `sys.setrecursionlimit`. An iterative DFS would avoid it, but the recursive version follows the assignment's hint directly.

6 Demo Results (`demo.ipynb`)

- **Gradient check.** For $f(a,b) = \tanh\big(\frac{ab+b^3}{a-4}\big) + e^{a/2} - 2/b$, every operator is used. The autograd gradients match central finite differences ($h = 10^{-6}$) to 6 decimal places: $\partial f/\partial a = 1.432677$, $\partial f/\partial b = 3.197680$.
- **Training.** `make_moons` (100 points, noise 0.1), `MLP(2,[16,16,1])` with 337 parameters, mean hinge loss with 10^{-4} L2, and SGD with the learning rate decaying from 1.0 to 0.1 over 100 steps. Loss drops from 0.866 to 0.014, and training accuracy reaches 100% by step 30.

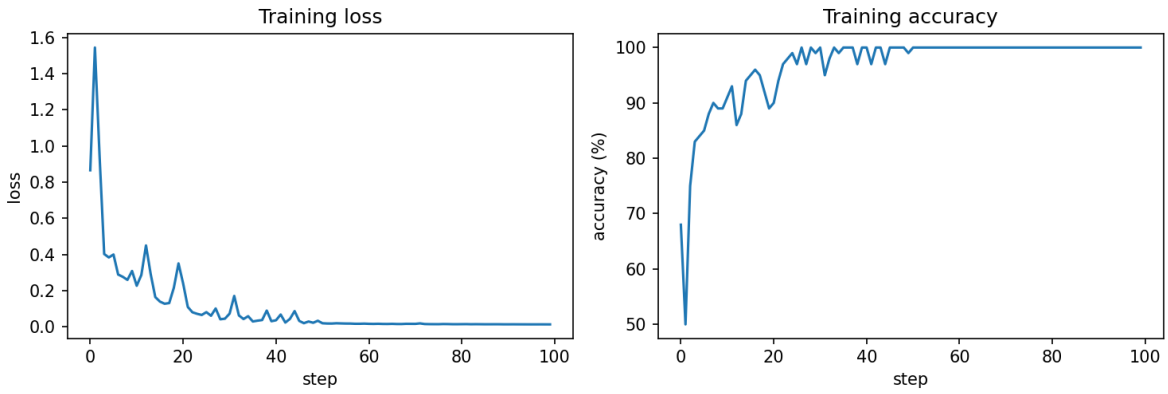


Figure 1: Training loss and accuracy.

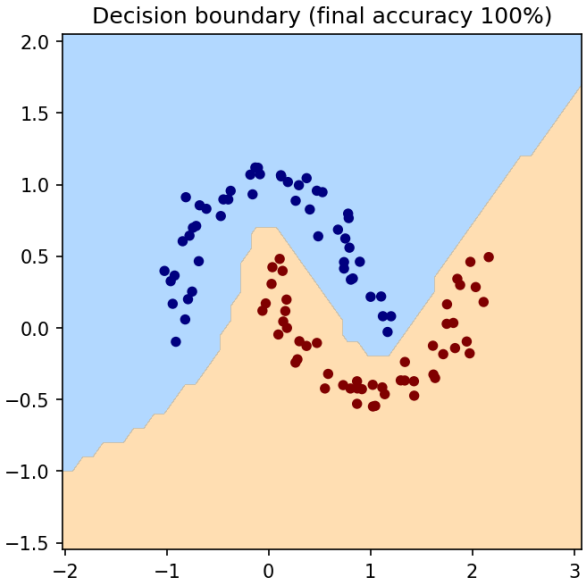


Figure 2: Learned decision boundary on the two-moons data.